

ACTL 3142

Assignment 2

z5690607
Simon Teng

Q1. a) The final classification model for rating_q1, a binary indicator for whether an Airbnb’s review score was in the top 25% quantile among neighbouring listings, is a logistic regression model with 15 predictors selected via LASSO regression by optimising AUC and then refitted as a GLM. By refitting, the shrinkage bias introduced by LASSO to penalise complexity is removed, thereby better representing the estimated changes in log-odds associated with each predictor.

Exponentiating the log-odds coefficients (see Appendix) gives the odds ratio (OR) for each predictor, representing the multiplicative change in the odds of a listing being classified in rating_q1. These can be interpreted as percentage changes, where ORs greater than one reflect positive associations with ratings such as superhost status, with the largest OR of 2.35. This indicates that listings operated by superhosts had a 135% increase in odds for ratings falling within the top 25% quantile. Conversely, OR less than one, as seen for predictors such as instantly bookable listings (0.73) was associated with 27% lower odds of being in rating_q1.

b) The initial model was a logistic model with a binomial distribution and logit link to ensure probability outputs between 0 and 1. Of the 52 variables in the raw data, 17 were excluded. Seven review score variables were excluded due to data leakage risk while a further ten variables were excluded due to data-quality concerns and redundancy arising from duplicate or engineered predictors. Notably, host neighbourhoods had 5,796 missing responses while response and acceptance rates were flagged for potential re-inclusion, given that each had a comparatively tolerable ~900 missing responses. Host tenure days, verification counts, and amenity counts were engineered from categorical lists into numerical counts. Finally, missing values in superhost status and host response time were recoded into structurally interpretable categories: “unevaluated” for superhost status and “non-responsive” for host response times. The data was then split 70/30 for training and testing, exposing flaws where exclusive property categories were not shared between the two sets, which warranted the creation of an “Other” category that collapsed types with fewer than 35 observations.

The resulting 37 predictors were assessed using both ridge and LASSO regression via cross-validation, creating models with near identical AUCs of 0.7377 and 0.7381. These penalty-based approaches were preferred over stepwise variable selection for their lower vulnerabilities to overfitting, producing more stable models when handling multicollinearity among predictors, which is highly likely given the correlated nature of Airbnb listing attributes. The near-identical AUCs indicated that the additional variables retained by ridge contributed limited predictive power, thereby lending support to the LASSO model on the grounds of parsimony and interpretability. Additionally, LASSO’s results were favoured for the automatic variable selection that provided additional guidance on further model development, notably shrinking 37 of 38 neighbourhood_cleansed levels and 3 of 4 host_response_time levels to zero.

Building upon this insight, the predictors selected by LASSO were refitted using various GLMs and chi-squared significance tests were performed to assess these borderline variables. While neighbour_cleansed returned a significant p-value of 0.02274, it was nevertheless excluded from the model due to the additional complexity of 37 degrees of freedom coupled with the increase to AUC from 0.7382 to 0.7388 associated with its exclusion. While host_response_time corresponded to a negligible AUC increase, the chi-squared test returned an insignificant p-value of 0.2107 thereby supporting its exclusion. These steps yielded the final 15 predictor model, slightly decreasing overall AUC for lower complexity and higher overall interpretability.

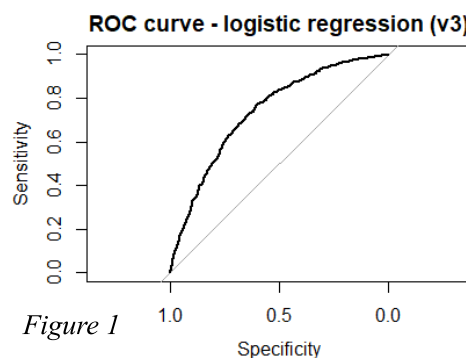


Figure 1

c) The final model achieved a test AUC of 0.735, as depicted in Figure 1, indicating meaningful discriminative ability beyond the 0.5 baseline expected under random guessing. While AUC reflects a model’s discriminative ability independent of classification decisions, the probability outputs must be converted into binary classifications using a chosen threshold. This threshold does not alter the overall model but can drastically alter classification performance, and should therefore be chosen based on the priorities of the model.

Threshold	Accuracy	Precision	Recall
0.25	66.3%	41.5%	72.7%
0.50	75.0%	55.6%	19.3%
0.75	74.4%	71.4%	2.41%

Figure 2

Thus, an optimistic threshold of 0.25 was adopted, approximately reflecting the 26% base rate of top-quartile listings in the training data, rather than the conventional 0.5 or the more conservative 0.75. This reflects the framing of a private investor screening properties for potential acquisition, where the costs of false negatives from overlooking promising listings exceed the costs of additional due diligence on false positives. Thus, a threshold optimised for recall, the proportion of true top-quartile listings correctly identified over precision, the proportion of predicted top-quartile listings that are correctly identified.

At this threshold, the model's recall indicates that 72.7% of top-rated listings were correctly identified with a precision of 41.5% and an overall accuracy of 66.3%. By comparison (Figure 2), the conventional 0.5 threshold prioritises precision (55.6%) over recall (19.3%) and is the only threshold that achieves an accuracy (75%) above a naive classifier predicting only low ratings (74%). While this accuracy limitation is a byproduct of the heavy class imbalance within the rating_q1, these statistics indicate more pressing concerns in that at this threshold, only one in five top-rated listings is correctly identified. At a 0.75 threshold, these issues are exacerbated with near-zero recall (2.4%) and a superficially high precision (71.4%) created on 21 positive predictions from a test sample of 2400 observations, making this model impractical. It is worth emphasising how these differences arise from the same probability model evaluated at different decision thresholds rather than three separate models.

Multicollinearity within the model was not a material concern as all factors had an adjusted generalised variance inflation factor of less than 1.45. The primary limitation is the model's linear structure, which limits its ability to capture non-linear relationships and higher-order interactions. This reflects a trade-off accepted by the model, prioritising the coefficient-level interpretability of GLMs at the expense of complexity and predictive power. Additionally, certain predictors introduce further caveats. Notably, the engineered "Other" category created for property types to enable train-test data splitting introduces ambiguity and makes it difficult to distinguish rare properties that may carry rating advantage. Superhost status also introduces a risk of data leakage, as Airbnb's own eligibility criteria may partly depend on high ratings, creating an association with rating_q1 that is not unidirectional.

Q2. The model, while tuned for locating promising listings, identifies several factors (Figure 3) investors can plausibly act upon to improve a listing and raise the odds of achieving a top-quartile rating. The strongest positive factor is superhost status with an OR of 2.35, although, as identified earlier, should be interpreted with caution. Airbnb's superhost criteria likely require high existing ratings and so a circular relationship may therefore be formed where the better ratings that are associated with superhost status likely stem from the same good practices. A more useful finding is the odds of achieving higher ratings when listings are not instantly bookable as these listings have an OR of 0.73. While initially counterintuitive as this presents an inconvenience for guests, it facilitates manual screening that allows hosts to better manage guest fit and expectations before a stay begins. This presents a low-cost, easily actionable change for hosts. Portfolio scale is also a significant factor, as hosts managing many listings demonstrated lower odds of achieving a top rating (host total listing counts demonstrated OR = 0.997), which is relevant when weighing self management against third-party property management.

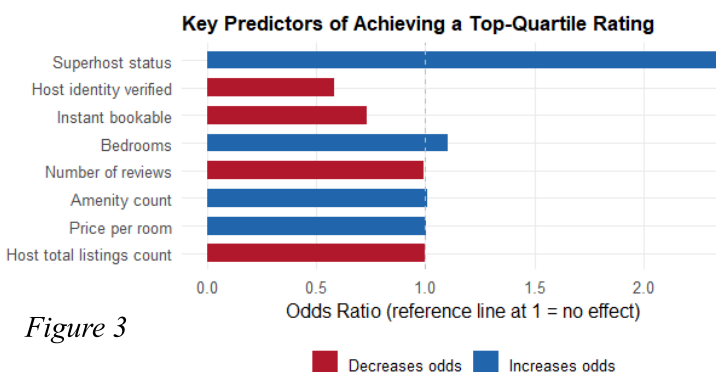


Figure 3

Beyond structural changes to the listing, bedrooms, amenities and prices per room were all positively associated with top-quartile ratings, supporting decisions centred around investment into property size, provisions and pricing. Another cautionary finding is with review counts, where listings with more accumulated reviews showed lower odds of a top rating (OR = 0.99 per review). This observation is consistent with the regression-to-the-mean phenomenon, where ratings will tend towards mean values as review counts increase. This, while not directly actionable, suggests to investors to not interpret large review counts as quality signals nor to overlook new, well-run listings with limited review history. Finally, host identity verification showed notable negative association with achieving top ratings (OR = 0.58), a counterintuitive result that may be caused by an experienced subset of hosts who began operation before the standardisation of verification practices. This warrants caution given there appear to be no clear causal mechanics connecting verification status to guest satisfaction.

Beyond structural changes to the listing, bedrooms, amenities and prices per room were all positively associated with top-quartile ratings, supporting decisions centred around investment into property size, provisions and pricing. Another cautionary finding is with review counts, where listings with more accumulated reviews showed lower odds of a top rating (OR = 0.99 per review). This observation is consistent with the regression-to-the-mean phenomenon, where ratings will tend towards mean values as review counts increase. This, while not directly actionable, suggests to investors to not interpret large review counts as quality signals nor to overlook new, well-run listings with limited review history. Finally, host identity verification showed notable negative association with achieving top ratings (OR = 0.58), a counterintuitive result that may be caused by an experienced subset of hosts who began operation before the standardisation of verification practices. This warrants caution given there appear to be no clear causal mechanics connecting verification status to guest satisfaction.

Q3. a) The final regression model for `price_per_room` was an extreme gradient-boosted (XGBM) tree model configured as a Gamma regression objective function with a maximum depth of 6, a learning rate of 0.05, and was reached after 337 boosting rounds, determined via early stopping under 10 fold cross-validation. The initially selected maximum depth and learning rates were then optimised via a 3 x 3 grid search where the Gamma objective was retained for consistency and to reflect the strictly positive right skewed nature of price per room.

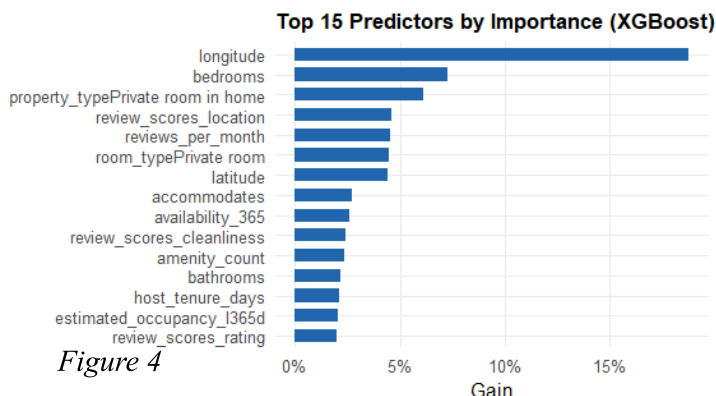


Figure 4

Prioritising predictive accuracy over interpretability, the XGBM's results are presented in Figure 4 in terms of variable importance. Longitude emerged as the greatest loss-reducing predictor, accounting for approximately 19% of total gain, roughly three times that of the next-ranked variable. This trend is consistent with Sydney's pronounced east-west price gradient that reflects the preference for coastal eastern suburbs. Bedrooms, property_types, particularly private rooms within houses, location score and reviews per month followed as the next most influential predictors.

b) Unlike the classification model, the seven review score ratings were retained as candidate predictors for the regression data as the leakage concern does not apply to price per room (PPR). The additional ten variables excluded from the classification dataset were excluded for similar reasons in the regression dataset and similar feature engineering was performed resulting in 44 candidate predictors. Furthermore, the host response and acceptance rate predictors were tested through staged reintroduction using identical training and test splits with and without the variables. To preserve data and prevent any distortion from other variable relationships, mean values were inputted for missing responses and ultimately yielded negligible improvements to MSE (4253.03 to 4252.52) under LASSO. Additionally, ridge MSE was unchanged despite its non-zero shrinking property confirming these variables' coefficients to be negligible.

Rather than transforming PPR through the logarithm to fit on ordinary least squares, PPR was modelled directly using a Gamma GLM with a log link. This was to avoid the retransformation bias when exponentiating fitted values from a $\log(\text{price_per_room})$ model stemming from Jensen's inequality, $E[\log Y] \leq \log(E[Y])$, which reflects the concavity of the logarithm function. This choice was consistent with the conclusions from the prior assignment where a Gamma-log-link GLM achieved materially better AIC than a Gaussian model of the logged response. Further experimentation yielded limited improvements, ultimately achieving a minimum MSE of 4231.53. This indicated a limit reached with linear models, forming a baseline model before proceeding to capture non-linear relationships through a generalised additive model (GAM) using smoothing splines.

The GAM, retaining the Gamma-log-link structure and using the LASSO predictor list as a starting point, produced reductions in test MSE to 3469.92, an 18% reduction over the ridge baseline and indicated that several predictor response relationships had non-linearity. Through diagnostic checking, the basis dimension parameter, $k = 10$, was found to be insufficient for the predictors latitude and host_tenure days which exhibited EDFs near 9. While iteratively raising the maximum degrees of freedom to 30 resolved host_tenure days, latitude remained problematic and overall MSE did not improve. As latitude and longitude jointly reflect locational data, a joint two-dimensional smooth was therefore created. However, diagnostic issues persisted and MSE increased. This suggested that some of Sydney's spatial price structure was already captured in neighbourhood_cleansed, once again indicating a limit to the minimum MSE achievable by this model.

To further improve on the GAM's non-linear, additive structure, an XGBM ensemble was fitted to test its automatic interaction detection. This achieved a test MSE of 3264.44, a 23% improvement over the ridge and 6% over the GAM. This being the largest gain across all models demonstrated how interaction effects between predictors extended beyond non-linearity in individual predictors to explain price per room. Further refinement was attempted on the initial tree through iterating along a 3 by 3 matrix of different potential maximum depths and learning rates but results indicated that the initial model, with a learning rate of 0.05 and a maximum depth of 6 was optimal.

c) The final model achieved a test MSE of 3264.44, representing a ~\$57 price deviation from actual listing prices. This is a 53% reduction from a naive model predicting the training mean for all listings (MSE ~7000). The consistent improvement across the model tiers, linear, additive non-linear and interaction capture, provides evidence that the underlying relationships are both non-linear and interactive, rather than improvements stemming from overfit.

<i>Figure 5</i> Model	Training RMSE	Test RMSE	Overfit gap	Test MSE	% Reduction vs Naive
LASSO (Gamma GLM)	69.53225	65.15037	-4.381878	4244.57	~39%
Ridge (Gamma GLM)	69.50958	64.97629	-4.533290	4221.92	~40%
GAM (Gamma, smoothing splines)	64.52750	58.90606	-5.621437	3469.92	~50%
XGBoost	31.99676	57.13524	25.138478	3264.44	~53%

However, overfitting is nonetheless the primary limitation of the XGBM model. Figure 5 compares the train and test RMSE across the best models from each model class. The regularised GLMs and the GAM showed small negative overfit gaps where the test RMSE was marginally below train RMSE. This is plausibly attributable to a higher concentration of outlier listings in the training split rather than poor generalisation, with a maximum PPR of 999 with a standard deviation of 88.2 compared to 787 and 83.6 in the test data. These small gaps are also consistent with the deliberately constrained complexity of these models through coefficient regularisation in the LASSO and ridge, and the smoothing of the GAM. By contrast, the XGBM has a large positive gap of \$25.1, indicating that it fits the training data considerably more closely than the unseen test data. While the XGBM achieves the lowest test RMSE overall and remains the preferred model, the model's flexibility is its limitation as its ability to capture non-linear interactions, while beneficial for error reduction, carries the genuine risk of overfitting. Thus, this test performance should be viewed as an estimate from a single train/test split rather than a robust measure of generalisation error.

Additionally, the XGBM model has limited interpretability because it cannot output interpretable coefficients like the GLM and GAM models. The XGBM provides variable importance which, while useful for identifying variable relevance, does not demonstrate the direction, magnitude or form of relationships. This is the explicit tradeoff made to minimise test MSE. As a final comment PPRs of Airbnb's in Sydney were found to follow a spatial price structure as evidenced by the GAM diagnostics, but had local variation not well captured by smooth or additive methods. The XGBoost's superior performance, driven substantially by longitude, suggests that tree-based partitioning better accommodates this structure and makes the XGBM a strong model for predicting PPR.

Q4. With additional time and resources, several realistic extensions could be made to the XGBM to address the limitations discussed in Q3. The \$25 train-test RMSE gap represents the main issue that could be addressed through further computation. With more processing power, a grid-search algorithm could be developed using a more conservative hyperparameter search that incorporates the unexplored regularisation terms lambda and alpha and the minimum child weight to create further splits and directly constrain the XGBM's overfit to training noise. Similarly, stricter early stopping rounds or joint tuning could be implemented with maximum depth alongside these regularisation parameters rather than isolating tree depth as was done here. This could meaningfully narrow this gap without sacrificing predictive performance. Furthermore, Q3 demonstrated how sampling artefacts emerged from the singular 70/30 train test split. Additional time and resources for repeated cross-validation in XGBM's hyperparameter tuning and evaluation would yield a model with a better split-independent estimate of generalisation error.

Longitude was identified as the XGBM's most significant predictor (19% gain), suggesting that Sydney's Airbnb pricing structure relies substantially on geographic signal. Thus, additional information and engineered features could improve the model's performance. While longitude reflects the general socio-economic status of Eastern-Sydney, additional noise exists within suburbs created by accessibility to services. Thus, explicit distance based predictors for proximity to desirable locations such as shopping centres, beaches, or other major landmarks could greatly improve spatial signal, thereby capturing Sydney's price geography more efficiently and with less reliance on raw coordinates.

Appendix:

Generative AI Usage

Generative AI usage in this assignment generally consisted of code troubleshooting and debugging, helped with interpretation of code results and quirks and provided report feedback on claims, results and general grammatical corrections.

AI usage was instrumental in explaining and improving my understanding of newer content – GAM and XGBM providing the relevant libraries and code explanations. AI was also used in these sections to help transform the Airbnb data into the form required for the packages used for these models. The function made for iteratively testing different hyperparameters in the XGBM was another instance of AI generating code. AI debugging was used when recollating the final XGBM model due to quirks with external pointers erasing the data matrixes imputed into the process, explaining what was occurring and the fix to rerun the data creation code.

Additionally, the cleaning of the AirbnbTest excel was done primarily by AI, providing suggestions for trouble shooting, error interpretations and analysis, and the suggestion of solutions.

Model Summary:

Q1 Final GLM model

```
> summary(glm_v3)
```

Call:

```
glm(formula = glm_formula_v3, family = "binomial", data = train_data_v3)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-3.915e-01	5.107e-01	-0.766	0.443389	
host_total_listings_count	-2.932e-03	5.592e-04	-5.242	1.58e-07	***
property_typeEntire condo	-6.064e-01	5.112e-01	-1.186	0.235453	
property_typeEntire cottage	-3.772e-01	6.321e-01	-0.597	0.550599	
property_typeEntire guest suite	-1.340e-01	5.046e-01	-0.266	0.790498	
property_typeEntire guesthouse	-3.283e-02	4.989e-01	-0.066	0.947531	
property_typeEntire home	-2.274e-01	4.798e-01	-0.474	0.635515	
property_typeEntire loft	-1.096e+00	7.560e-01	-1.450	0.147058	
property_typeEntire rental unit	-4.897e-01	4.709e-01	-1.040	0.298330	
property_typeEntire serviced apartment	-1.229e+00	5.755e-01	-2.135	0.032729	*
property_typeEntire townhouse	-9.242e-01	5.487e-01	-1.684	0.092123	.
property_typeEntire villa	7.384e-02	6.636e-01	0.111	0.911402	
property_typeOther	-4.714e-02	5.175e-01	-0.091	0.927426	
property_typePrivate room in condo	6.521e-01	5.802e-01	1.124	0.261069	
property_typePrivate room in guest suite	-1.017e+00	9.016e-01	-1.128	0.259512	
property_typePrivate room in home	-2.606e-01	4.826e-01	-0.540	0.589120	
property_typePrivate room in rental unit	2.394e-01	4.934e-01	0.485	0.627469	
property_typePrivate room in townhouse	-4.280e-01	6.126e-01	-0.699	0.484771	
property_typeRoom in aparthotel	-1.905e+00	7.772e-01	-2.451	0.014238	*
property_typeRoom in boutique hotel	-1.874e+00	1.155e+00	-1.622	0.104705	
property_typeRoom in hotel	-2.123e+00	8.702e-01	-2.440	0.014689	*
property_typeTiny home	-1.515e-01	7.670e-01	-0.197	0.843457	
maximum_nights	-5.494e-04	9.274e-05	-5.924	3.15e-09	***
availability_90	-7.935e-03	1.168e-03	-6.794	1.09e-11	***
number_of_reviews_l30d	1.808e-01	2.472e-02	7.316	2.56e-13	***
instant_bookable	-3.155e-01	7.800e-02	-4.045	5.24e-05	***
amenity_count	7.668e-03	2.415e-03	3.175	0.001500	**
host_is_superhostnot_evaluated	4.298e-01	1.782e-01	2.412	0.015852	*
host_is_superhostt	8.560e-01	7.710e-02	11.103	< 2e-16	***
host_identity_verifiedt	-5.385e-01	1.661e-01	-3.242	0.001186	**

longitude	-1.080e+00	3.074e-01	-3.514	0.000441	***
bedrooms	1.000e-01	3.928e-02	2.546	0.010890	*
minimum_minimum_nights	6.594e-03	1.362e-03	4.842	1.29e-06	***
number_of_reviews	-7.512e-03	8.199e-04	-9.162	< 2e-16	***
estimated_occupancy_l365d	-4.102e-03	5.631e-04	-7.285	3.21e-13	***
price_per_room	4.108e-03	4.216e-04	9.742	< 2e-16	***

Signif. codes:	0	'***'	0.001	'**'	0.01
	'*'	0.05	'.'	0.1	' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 6412 on 5599 degrees of freedom
 Residual deviance: 5574 on 5564 degrees of freedom
 AIC: 5646

Number of Fisher Scoring iterations: 5

Q3. Final Models:

Ridge:

> ridge_coefs_final

	lambda.min
(Intercept)	-1.331467e+02
host_response_timenever_responds	1.098975e-01
host_response_timewithin a day	3.693683e-02
host_response_timewithin a few hours	1.919817e-02
host_response_timewithin an hour	5.432618e-03
host_is_superhostnot_evaluated	-4.357137e-02
host_is_superhostt	3.168167e-02
host_listings_count	-3.518809e-04
host_total_listings_count	1.137989e-04
host_has_profile_pict	4.955707e-02
host_identity_verifiedt	7.876034e-03
neighbourhood_cleansedAuburn	-2.371188e-02
neighbourhood_cleansedBankstown	-1.387128e-01
neighbourhood_cleansedBlacktown	-1.634072e-01
neighbourhood_cleansedBotany Bay	-7.416424e-02
neighbourhood_cleansedBurwood	4.211148e-02
neighbourhood_cleansedCamden	3.376481e-01
neighbourhood_cleansedCampbelltown	1.601598e-01
neighbourhood_cleansedCanada Bay	4.444818e-02
neighbourhood_cleansedCanterbury	-2.162937e-01
neighbourhood_cleansedCity Of Kogarah	-1.148454e-01
neighbourhood_cleansedFairfield	-2.852525e-01
neighbourhood_cleansedHolroyd	-1.680755e-01
neighbourhood_cleansedHornsby	-1.750621e-01
neighbourhood_cleansedHunters Hill	-1.121856e-01
neighbourhood_cleansedHurstville	-1.741644e-01
neighbourhood_cleansedKu-Ring-Gai	-2.092447e-01
neighbourhood_cleansedLane Cove	-5.528593e-02
neighbourhood_cleansedLeichhardt	6.028996e-03
neighbourhood_cleansedLiverpool	-1.322549e-01
neighbourhood_cleansedManly	1.055881e-01
neighbourhood_cleansedMarrickville	-5.387990e-03
neighbourhood_cleansedMosman	1.438061e-01
neighbourhood_cleansedNorth Sydney	1.592336e-01
neighbourhood_cleansedParramatta	-1.240528e-01
neighbourhood_cleansedPenrith	1.167351e-01

neighbourhood_cleansedPittwater	2.704518e-02
neighbourhood_cleansedRandwick	9.976722e-02
neighbourhood_cleansedRockdale	-9.689760e-02
neighbourhood_cleansedRyde	-1.057364e-01
neighbourhood_cleansedStrathfield	-1.068085e-01
neighbourhood_cleansedSutherland Shire	2.075606e-01
neighbourhood_cleansedSydney	1.462339e-01
neighbourhood_cleansedThe Hills Shire	-1.075121e-01
neighbourhood_cleansedWarringah	-6.546690e-02
neighbourhood_cleansedWaverley	1.352307e-01
neighbourhood_cleansedWilloughby	-1.301649e-01
neighbourhood_cleansedWoollahra	1.428276e-01
latitude	7.697739e-01
longitude	1.081660e+00
property_typeEntire condo	-3.838423e-02
property_typeEntire cottage	1.087085e-01
property_typeEntire guest suite	-5.423581e-02
property_typeEntire guesthouse	-1.545260e-02
property_typeEntire home	8.777179e-02
property_typeEntire loft	-3.011767e-03
property_typeEntire rental unit	3.981461e-03
property_typeEntire serviced apartment	1.379541e-01
property_typeEntire townhouse	-8.883407e-02
property_typeEntire villa	2.285780e-01
property_typeOther	5.536801e-02
property_typePrivate room in condo	-1.731462e-01
property_typePrivate room in guest suite	1.021979e-01
property_typePrivate room in home	-2.361295e-01
property_typePrivate room in rental unit	-1.147204e-01
property_typePrivate room in townhouse	-1.369761e-01
property_typeRoom in aparthotel	4.021013e-01
property_typeRoom in boutique hotel	4.409019e-01
property_typeRoom in hotel	3.588371e-01
property_typeTiny home	1.003113e-02
room_typePrivate room	-2.942891e-01
accommodates	2.498825e-02
bathrooms	1.059566e-01
bedrooms	-1.935606e-01
beds	-2.576002e-04
minimum_nights	1.175712e-03
maximum_nights	3.925647e-05
minimum_minimum_nights	-4.011068e-04
maximum_minimum_nights	1.295392e-04
minimum_maximum_nights	-1.479082e-04
maximum_maximum_nights	1.055940e-04
minimum_nights_avg_ntm	-1.745667e-03
has_availabilityt	-3.753529e-01
availability_30	6.220189e-03
availability_60	-3.156443e-04
availability_90	-7.933142e-04
availability_365	2.467959e-04
number_of_reviews	-6.209413e-05
number_of_reviews_ltm	1.299891e-03
number_of_reviews_l30d	-2.074293e-02
estimated_occupancy_l365d	-6.898641e-04
review_scores_rating	3.311725e-02
review_scores_accuracy	8.669864e-02
review_scores_cleanliness	1.796418e-01
review_scores_checkin	-2.335770e-02

review_scores_communication	-8.691057e-02
review_scores_location	1.718326e-01
review_scores_value	-1.706671e-01
instant_bookable	-1.823932e-03
reviews_per_month	-2.214089e-02
rating_q1	1.360096e-01
host_tenure_days	1.322815e-05
verification_count	-5.346509e-04
amenity_count	2.038023e-03

LASSO:

```
> lasso_coefs_final
```

	lambda.min
(Intercept)	-1.903147e+02
host_response_timenever_responds	1.045999e-01
host_response_timewithin a day	2.904067e-02
host_response_timewithin a few hours	1.293161e-02
host_response_timewithin an hour	0.000000e+00
host_is_superhostnot_evaluated	-4.398299e-02
host_is_superhostt	2.893224e-02
host_listings_count	-3.152438e-04
host_total_listings_count	7.325696e-05
host_has_profile_pict	4.424625e-02
host_identity_verifiedt	5.172681e-03
neighbourhood_cleansedAuburn	5.504473e-02
neighbourhood_cleansedBankstown	-3.858207e-02
neighbourhood_cleansedBlacktown	-7.468678e-03
neighbourhood_cleansedBotany Bay	-4.000268e-02
neighbourhood_cleansedBurwood	1.028617e-01
neighbourhood_cleansedCamden	5.537574e-01
neighbourhood_cleansedCampbelltown	3.330779e-01
neighbourhood_cleansedCanada Bay	1.092138e-01
neighbourhood_cleansedCanterbury	-1.434248e-01
neighbourhood_cleansedCity Of Kogarah	-4.560244e-02
neighbourhood_cleansedFairfield	-1.528228e-01
neighbourhood_cleansedHolroyd	-5.173870e-02
neighbourhood_cleansedHornsby	-1.236744e-01
neighbourhood_cleansedHunters Hill	-4.692863e-02
neighbourhood_cleansedHurstville	-9.538550e-02
neighbourhood_cleansedKu-Ring-Gai	-1.673777e-01
neighbourhood_cleansedLane Cove	-5.341067e-03
neighbourhood_cleansedLeichhardt	4.311390e-02
neighbourhood_cleansedLiverpool	1.189897e-02
neighbourhood_cleansedManly	1.018829e-01
neighbourhood_cleansedMarrickville	3.385064e-02
neighbourhood_cleansedMosman	1.585827e-01
neighbourhood_cleansedNorth Sydney	1.850434e-01
neighbourhood_cleansedParramatta	-2.391160e-02
neighbourhood_cleansedPenrith	3.262572e-01
neighbourhood_cleansedPittwater	-1.547246e-03
neighbourhood_cleansedRandwick	1.159010e-01
neighbourhood_cleansedRockdale	-3.980746e-02
neighbourhood_cleansedRyde	-3.919797e-02
neighbourhood_cleansedStrathfield	-2.665601e-02
neighbourhood_cleansedSutherland Shire	2.745882e-01
neighbourhood_cleansedSydney	1.728847e-01
neighbourhood_cleansedThe Hills Shire	0.000000e+00
neighbourhood_cleansedWarringah	-7.093434e-02
neighbourhood_cleansedWaverley	1.419328e-01
neighbourhood_cleansedWilloughby	-9.795585e-02
neighbourhood_cleansedWoollahra	1.584250e-01
latitude	8.225845e-01
longitude	1.471237e+00
property_typeEntire condo	-3.651695e-02
property_typeEntire cottage	1.089346e-01
property_typeEntire guest suite	-5.576605e-02
property_typeEntire guesthouse	-1.402727e-02
property_typeEntire home	9.618353e-02
property_typeEntire loft	0.000000e+00

property_typeEntire rental unit	0.000000e+00
property_typeEntire serviced apartment	1.299093e-01
property_typeEntire townhouse	-8.033628e-02
property_typeEntire villa	2.193433e-01
property_typeOther	1.072116e-01
property_typePrivate room in condo	-7.558654e-02
property_typePrivate room in guest suite	1.844751e-01
property_typePrivate room in home	-1.401785e-01
property_typePrivate room in rental unit	-1.411181e-02
property_typePrivate room in townhouse	-3.494044e-02
property_typeRoom in aparthotel	4.488133e-01
property_typeRoom in boutique hotel	5.226807e-01
property_typeRoom in hotel	4.428978e-01
property_typeTiny home	3.370370e-03
room_typePrivate room	-3.905381e-01
accommodates	3.389581e-02
bathrooms	1.173845e-01
bedrooms	-2.227257e-01
beds	0.000000e+00
minimum_nights	1.257069e-03
maximum_nights	3.482187e-05
minimum_minimum_nights	-7.194988e-05
maximum_minimum_nights	1.343969e-04
minimum_maximum_nights	-1.662821e-04
maximum_maximum_nights	1.233154e-04
minimum_nights_avg_ntm	-2.122855e-03
has_availabilityt	-3.337248e-01
availability_30	6.269575e-03
availability_60	-1.231292e-04
availability_90	-9.327723e-04
availability_365	2.470729e-04
number_of_reviews	-6.292281e-05
number_of_reviews_ltm	1.413799e-03
number_of_reviews_l30d	-2.049150e-02
estimated_occupancy_l365d	-7.071556e-04
review_scores_rating	2.213822e-02
review_scores_accuracy	8.773957e-02
review_scores_cleanliness	1.903985e-01
review_scores_checkin	-1.803970e-02
review_scores_communication	-8.484580e-02
review_scores_location	1.710458e-01
review_scores_value	-1.788894e-01
instant_bookablet	-9.942626e-05
reviews_per_month	-2.349957e-02
rating_q1	1.393678e-01
host_tenure_days	1.271353e-05
verification_count	0.000000e+00
amenity_count	2.095635e-03

GAM:

```
> summary(Q3_final_gam)
```

Family: Gamma

Link function: log

Formula:

```
price_per_room ~ host_response_time + host_is_superhost + s(host_listings_count,
  k = 10) + s(host_total_listings_count, k = 10) + host_has_profile_pic +
  host_identity_verified + neighbourhood_cleansed + s(latitude,
  k = 10) + s(longitude, k = 10) + property_type + room_type +
  s(accommodates, k = 10) + s(bathrooms, k = 10) + s.bedrooms,
  k = 10) + s(minimum_nights, k = 10) + s(maximum_nights, k = 10) +
  s(availability_90, k = 10) + s(number_of_reviews, k = 10) +
  s(number_of_reviews_ltm, k = 10) + s(number_of_reviews_l30d,
  k = 10) + s(estimated_occupancy_l365d, k = 10) + s(review_scores_rating,
  k = 10) + s(review_scores_accuracy, k = 10) + s(review_scores_cleanliness,
  k = 10) + s(review_scores_checkin, k = 10) + s(review_scores_communication,
  k = 10) + s(review_scores_location, k = 10) + s(review_scores_value,
  k = 10) + instant_bookable + s(reviews_per_month, k = 10) +
  s(host_tenure_days, k = 10) + s(amenity_count, k = 10)
```

Parametric coefficients:

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	4.681e+00	1.271e-01	36.843	< 2e-16	***
host_response_time	never responds	1.363e-01	5.422e-02	2.514	0.011967 *
host_response_time	within a day	9.284e-02	5.743e-02	1.616	0.106065
host_response_time	within a few hours	8.605e-02	5.568e-02	1.546	0.122275
host_response_time	within an hour	8.776e-02	5.426e-02	1.617	0.105832
host_is_superhost	not evaluated	-2.957e-02	2.935e-02	-1.008	0.313604
host_is_superhost	t	2.609e-02	1.359e-02	1.920	0.054901 .
host_has_profile_pic	t	4.131e-02	4.296e-02	0.962	0.336251
host_identity_verified	t	1.944e-02	2.914e-02	0.667	0.504761
neighbourhood_cleansed	Auburn	-5.362e-03	8.047e-02	-0.067	0.946882
neighbourhood_cleansed	Bankstown	7.198e-02	9.142e-02	0.787	0.431117
neighbourhood_cleansed	Blacktown	5.260e-02	1.515e-01	0.347	0.728357
neighbourhood_cleansed	Botany Bay	1.411e-01	8.484e-02	1.663	0.096372 .
neighbourhood_cleansed	Burwood	6.899e-02	8.768e-02	0.787	0.431385
neighbourhood_cleansed	Camden	1.372e-01	1.840e-01	0.746	0.455800
neighbourhood_cleansed	Campbelltown	1.822e-01	1.586e-01	1.149	0.250569
neighbourhood_cleansed	Canada Bay	8.906e-02	8.021e-02	1.110	0.266891
neighbourhood_cleansed	Canterbury	6.508e-02	8.564e-02	0.760	0.447309
neighbourhood_cleansed	City Of Kogarah	6.117e-02	1.078e-01	0.567	0.570603
neighbourhood_cleansed	Fairfield	-1.862e-01	1.207e-01	-1.543	0.122913
neighbourhood_cleansed	Holroyd	5.666e-03	1.266e-01	0.045	0.964320
neighbourhood_cleansed	Hornsby	-9.358e-03	1.119e-01	-0.084	0.933344
neighbourhood_cleansed	Hunters Hill	-1.580e-01	1.701e-01	-0.929	0.352946
neighbourhood_cleansed	Hurstville	6.520e-02	1.142e-01	0.571	0.567929
neighbourhood_cleansed	Ku-Ring-Gai	-2.856e-02	1.090e-01	-0.262	0.793309
neighbourhood_cleansed	Lane Cove	-6.397e-02	1.054e-01	-0.607	0.543917
neighbourhood_cleansed	Leichhardt	7.776e-03	7.059e-02	0.110	0.912297
neighbourhood_cleansed	Liverpool	1.066e-01	1.311e-01	0.813	0.416316
neighbourhood_cleansed	Manly	1.691e-01	1.083e-01	1.562	0.118354
neighbourhood_cleansed	Marrickville	1.039e-01	6.766e-02	1.536	0.124629
neighbourhood_cleansed	Mosman	1.066e-01	9.840e-02	1.083	0.278911
neighbourhood_cleansed	North Sydney	1.311e-01	8.136e-02	1.611	0.107236
neighbourhood_cleansed	Parramatta	5.103e-02	1.047e-01	0.487	0.626045
neighbourhood_cleansed	Penrith	5.629e-02	1.867e-01	0.301	0.763106
neighbourhood_cleansed	Pittwater	1.025e-01	1.336e-01	0.767	0.443115

neighbourhood_cleansedRandwick	2.445e-01	8.251e-02	2.963	0.003061	**
neighbourhood_cleansedRockdale	1.555e-01	8.031e-02	1.937	0.052829	.
neighbourhood_cleansedRyde	3.284e-02	9.366e-02	0.351	0.725879	
neighbourhood_cleansedStrathfield	-1.019e-01	8.548e-02	-1.192	0.233461	
neighbourhood_cleansedSutherland Shire	6.120e-02	1.173e-01	0.522	0.601889	
neighbourhood_cleansedSydney	1.839e-01	6.798e-02	2.705	0.006862	**
neighbourhood_cleansedThe Hills Shire	1.635e-01	1.306e-01	1.252	0.210649	
neighbourhood_cleansedWarringah	3.143e-02	1.169e-01	0.269	0.788094	
neighbourhood_cleansedWaverley	1.467e-01	8.110e-02	1.809	0.070557	.
neighbourhood_cleansedWilloughby	-4.024e-02	9.995e-02	-0.403	0.687238	
neighbourhood_cleansedWoollahra	1.410e-01	8.062e-02	1.749	0.080311	.
property_typeEntire condo	2.211e-02	8.681e-02	0.255	0.798954	
property_typeEntire cottage	8.775e-02	1.011e-01	0.868	0.385438	
property_typeEntire guest suite	-8.821e-02	8.440e-02	-1.045	0.295970	
property_typeEntire guesthouse	1.316e-05	8.411e-02	0.000	0.999875	
property_typeEntire home	1.339e-01	8.138e-02	1.645	0.099966	.
property_typeEntire loft	4.911e-02	1.124e-01	0.437	0.662219	
property_typeEntire rental unit	6.175e-02	8.039e-02	0.768	0.442440	
property_typeEntire serviced apartment	1.353e-01	8.967e-02	1.509	0.131446	
property_typeEntire townhouse	7.005e-03	8.942e-02	0.078	0.937568	
property_typeEntire villa	2.259e-01	1.075e-01	2.102	0.035572	*
property_typeOther	1.122e-01	9.594e-02	1.169	0.242351	
property_typePrivate room in condo	-1.304e-01	1.199e-01	-1.088	0.276853	
property_typePrivate room in guest suite	1.618e-01	1.454e-01	1.113	0.265791	
property_typePrivate room in home	-1.407e-01	1.044e-01	-1.347	0.177907	
property_typePrivate room in rental unit	-5.537e-02	1.064e-01	-0.520	0.602738	
property_typePrivate room in townhouse	-5.280e-02	1.183e-01	-0.446	0.655272	
property_typeRoom in aparthotel	4.684e-01	1.086e-01	4.313	1.64e-05	***
property_typeRoom in boutique hotel	4.805e-01	1.313e-01	3.660	0.000255	***
property_typeRoom in hotel	3.363e-01	1.134e-01	2.966	0.003032	**
property_typeTiny home	2.875e-02	1.288e-01	0.223	0.823295	
room_typePrivate room	-3.705e-01	6.673e-02	-5.552	2.95e-08	***
instant_bookable	6.340e-03	1.249e-02	0.508	0.611768	

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

	edf	Ref.df	F	p-value	
s(host_listings_count)	1.969	2.378	0.302	0.787114	
s(host_total_listings_count)	8.148	8.544	2.789	0.004917	**
s(latitude)	8.583	8.951	10.161	< 2e-16	***
s(longitude)	4.638	5.970	4.815	7.03e-05	***
s(accommodates)	5.695	6.720	27.542	< 2e-16	***
s(bathrooms)	4.498	5.538	22.719	< 2e-16	***
s(bedrooms)	8.999	9.000	92.918	< 2e-16	***
s(minimum_nights)	7.404	8.342	5.145	3.09e-06	***
s(maximum_nights)	8.637	8.940	3.623	0.000123	***
s(availability_90)	5.154	6.229	13.712	< 2e-16	***
s(number_of_reviews)	1.250	1.458	2.767	0.094891	.
s(number_of_reviews_ltm)	5.845	6.814	1.023	0.472310	
s(number_of_reviews_l30d)	4.408	5.254	11.789	< 2e-16	***
s(estimated_occupancy_l365d)	2.779	3.518	2.241	0.073912	.
s(review_scores_rating)	5.453	6.407	6.656	1.23e-06	***
s(review_scores_accuracy)	2.277	2.943	2.531	0.069579	.
s(review_scores_cleanliness)	2.050	2.651	15.139	< 2e-16	***
s(review_scores_checkin)	1.514	1.879	1.125	0.406584	
s(review_scores_communication)	1.002	1.003	7.593	0.005844	**
s(review_scores_location)	6.139	7.154	6.605	< 2e-16	***
s(review_scores_value)	2.881	3.694	10.075	3.23e-07	***

```
s(reviews_per_month)      4.097  5.150  8.162  < 2e-16 ***
s(host_tenure_days)       3.864  4.813  1.420  0.225671
s(amenity_count)          4.215  5.194  9.366  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.447   Deviance explained = 60.2%
GCV = 0.11609   Scale est. = 0.13953   n = 5600
```

XGBM:

```
> cat("XGBoost final model specification:\n")
```

```
XGBoost final model specification:
```

```
> cat("Objective:", xgb_params$objective, "\n")
```

```
Objective: reg:gamma
```

```
> cat("max_depth:", xgb_params$max_depth, "\n")
```

```
max_depth: 6
```

```
> cat("eta:", xgb_params$eta, "\n")
```

```
eta: 0.05
```

```
> cat("subsample:", xgb_params$subsample, "\n")
```

```
subsample: 0.8
```

```
> cat("colsample_bytree:", xgb_params$colsample_bytree, "\n")
```

```
colsample_bytree: 0.8
```

```
> cat("nrounds:", best_nrounds_Q3, "\n")
```

```
nrounds: 337
```

```
> importance_Q3_full
```

	Feature <char>	Gain <num>	Cover <num>	Frequency <num>
1:	longitude	1.873997e-01	7.080766e-02	5.814752e-02
2:	bedrooms	7.244158e-02	6.957300e-02	2.635688e-02
3:	property_typePrivate room in home	6.142988e-02	5.750166e-03	3.934795e-03
4:	review_scores_location	4.608283e-02	2.416568e-02	2.910499e-02
5:	reviews_per_month	4.559280e-02	3.853925e-02	3.766161e-02
6:	room_typePrivate room	4.504909e-02	1.218645e-02	6.682906e-03
7:	latitude	4.447762e-02	5.957949e-02	6.439323e-02
8:	accommodates	2.744713e-02	3.158682e-02	2.535757e-02
9:	availability_365	2.573463e-02	3.536186e-02	4.484417e-02
10:	review_scores_cleanliness	2.401564e-02	2.898729e-02	2.760602e-02
11:	amenity_count	2.362884e-02	4.306464e-02	4.303291e-02
12:	bathrooms	2.177567e-02	2.623870e-02	1.705078e-02
13:	host_tenure_days	2.117853e-02	3.748881e-02	4.515645e-02
14:	estimated_occupancy_l365d	2.038297e-02	1.620073e-02	2.067329e-02
15:	review_scores_rating	1.969636e-02	2.523967e-02	2.616951e-02
16:	number_of_reviews_l30d	1.771473e-02	1.288484e-02	7.869590e-03
17:	availability_30	1.699090e-02	2.045393e-02	2.548248e-02
18:	availability_60	1.519905e-02	1.364221e-02	2.317157e-02
19:	review_scores_value	1.493152e-02	2.479691e-02	2.741865e-02
20:	number_of_reviews	1.472807e-02	2.291974e-02	3.322716e-02
21:	host_total_listings_count	1.380273e-02	2.001257e-02	2.816813e-02
22:	minimum_nights_avg_ntm	1.371840e-02	1.747988e-02	1.948660e-02
23:	availability_90	1.342919e-02	2.352486e-02	2.148523e-02
24:	number_of_reviews_ltm	1.261910e-02	1.237721e-02	1.873712e-02
25:	host_listings_count	1.251269e-02	1.983137e-02	3.372681e-02
26:	minimum_maximum_nights	1.238673e-02	2.468011e-02	1.598901e-02
27:	review_scores_accuracy	1.116659e-02	1.552712e-02	1.948660e-02
28:	minimum_nights	1.090462e-02	7.554128e-03	1.473987e-02
29:	property_typePrivate room in rental unit	1.089805e-02	3.377964e-03	2.498282e-03
30:	review_scores_checkin	8.988043e-03	1.199402e-02	2.167260e-02
31:	maximum_nights	8.531072e-03	1.304291e-02	2.029854e-02
32:	maximum_minimum_nights	7.922818e-03	9.084730e-03	1.455250e-02
33:	review_scores_communication	7.122964e-03	1.586713e-02	1.811255e-02
34:	minimum_minimum_nights	6.941236e-03	1.217719e-02	1.124227e-02
35:	property_typeEntire home	6.908356e-03	7.745329e-03	4.871651e-03
36:	beds	6.639912e-03	1.312596e-02	1.261633e-02
37:	property_typeEntire rental unit	5.077602e-03	4.071277e-03	6.433077e-03
38:	neighbourhood_cleansedSydney	4.561295e-03	4.064438e-03	5.433764e-03

39:	instant_bookable	4.224311e-03	1.283370e-03	4.434451e-03
40:	property_typeRoom in hotel	4.094688e-03	2.885343e-04	5.621135e-04
41:	property_typeEntire serviced apartment	3.767406e-03	3.417739e-03	2.185997e-03
42:	maximum_maximum_nights	3.340728e-03	5.334770e-03	6.870277e-03
43:	neighbourhood_cleansedWarringah	3.091041e-03	8.590658e-03	3.247767e-03
44:	property_typeRoom in aparthotel	2.949900e-03	5.035282e-03	1.623884e-03
45:	host_response_timenever_responds	2.456992e-03	4.661202e-03	7.682218e-03
46:	property_typeOther	2.214340e-03	2.660135e-03	2.623197e-03
47:	verification_count	2.210985e-03	2.346831e-03	5.121479e-03
48:	host_is_superhost	1.879740e-03	1.661879e-03	6.932734e-03
49:	neighbourhood_cleansedNorth Sydney	1.659354e-03	1.831200e-03	2.873025e-03
50:	host_response_timewithin an hour	1.657416e-03	9.252797e-04	6.058335e-03
51:	rating_q1	1.604983e-03	1.783308e-03	1.811255e-03
52:	property_typeRoom in boutique hotel	1.453320e-03	2.594525e-03	5.621135e-04
53:	host_response_timewithin a few hours	1.442929e-03	1.204598e-03	6.120792e-03
54:	neighbourhood_cleansedPittwater	1.186716e-03	8.213093e-04	1.561427e-03
55:	neighbourhood_cleansedWaverley	1.179915e-03	1.433454e-03	3.060396e-03
56:	neighbourhood_cleansedCanada Bay	1.047052e-03	5.742952e-03	1.998626e-03
57:	neighbourhood_cleansedMarrickville	1.024788e-03	2.464304e-03	2.061083e-03
58:	neighbourhood_cleansedBurwood	9.480173e-04	3.357907e-03	1.374055e-03
59:	property_typeEntire guest suite	9.127454e-04	6.500123e-03	1.623884e-03
60:	neighbourhood_cleansedManly	8.962101e-04	4.659723e-03	2.810568e-03
61:	host_response_timewithin a day	8.863853e-04	7.230904e-04	4.496908e-03
62:	neighbourhood_cleansedSutherland Shire	8.436316e-04	2.593772e-04	1.498969e-03
63:	neighbourhood_cleansedBotany Bay	7.509770e-04	4.303762e-03	1.686341e-03
64:	property_typeEntire townhouse	7.477183e-04	3.498816e-03	1.561427e-03
65:	property_typePrivate room in townhouse	6.956497e-04	6.092192e-03	1.249141e-03
66:	property_typeEntire villa	6.813656e-04	1.878531e-03	8.743989e-04
67:	property_typeEntire condo	5.833321e-04	1.170066e-03	1.436512e-03
68:	host_has_profile_pict	5.659284e-04	3.923637e-03	1.811255e-03
69:	property_typeEntire guesthouse	5.452347e-04	6.320202e-03	1.748798e-03
70:	property_typePrivate room in condo	5.265349e-04	6.757570e-03	1.124227e-03
71:	neighbourhood_cleansedWilloughby	5.126124e-04	5.165242e-03	1.748798e-03
72:	property_typeTiny home	5.015513e-04	4.322141e-03	1.311598e-03
73:	neighbourhood_cleansedAuburn	4.589028e-04	9.002162e-04	9.993130e-04
74:	host_identity_verified	4.540437e-04	3.839264e-04	1.811255e-03
75:	neighbourhood_cleansedRandwick	4.425194e-04	1.758654e-03	1.561427e-03
76:	neighbourhood_cleansedHolroyd	4.278360e-04	9.060438e-04	9.993130e-04
77:	neighbourhood_cleansedMosman	4.034353e-04	2.220488e-03	1.249141e-03
78:	host_is_superhostnot_evaluated	3.582483e-04	2.100595e-03	1.998626e-03
79:	neighbourhood_cleansedBlacktown	3.218782e-04	5.751058e-04	5.621135e-04
80:	neighbourhood_cleansedCamden	3.195613e-04	2.340288e-03	8.743989e-04
81:	neighbourhood_cleansedParramatta	2.990551e-04	4.502374e-04	1.061770e-03
82:	neighbourhood_cleansedKu-Ring-Gai	2.988953e-04	7.767205e-05	3.747424e-04
83:	neighbourhood_cleansedLeichhardt	2.917229e-04	1.806324e-03	1.561427e-03
84:	neighbourhood_cleansedFairfield	2.896723e-04	4.243854e-03	7.494847e-04
85:	neighbourhood_cleansedCampbelltown	2.704232e-04	4.093323e-04	2.498282e-04
86:	neighbourhood_cleansedLane Cove	2.700510e-04	2.657923e-03	8.119418e-04
87:	neighbourhood_cleansedWoollahra	2.668504e-04	3.429975e-04	9.993130e-04
88:	neighbourhood_cleansedBankstown	2.550857e-04	1.706940e-03	5.621135e-04
89:	neighbourhood_cleansedRyde	2.391624e-04	1.331875e-03	1.124227e-03
90:	neighbourhood_cleansedPenrith	2.371682e-04	5.389572e-04	4.996565e-04
91:	neighbourhood_cleansedStrathfield	2.361634e-04	3.439623e-03	9.993130e-04
92:	neighbourhood_cleansedThe Hills Shire	2.271177e-04	3.212078e-05	1.498969e-03
93:	neighbourhood_cleansedHornsby	2.152315e-04	8.933875e-04	1.061770e-03
94:	property_typeEntire cottage	1.903958e-04	1.132817e-03	6.870277e-04
95:	property_typeEntire loft	3.877388e-05	4.699142e-04	1.873712e-04
96:	neighbourhood_cleansedCanterbury	3.685598e-05	7.626523e-04	3.122853e-04
97:	neighbourhood_cleansedLiverpool	1.817317e-05	5.743716e-06	6.245706e-05

98:	neighbourhood_cleansedRockdale	1.475362e-05	7.574260e-06	4.371994e-04
99:	neighbourhood_cleansedHunters Hill	4.597876e-06	4.849776e-04	6.245706e-05
	Feature	Gain	Cover	Frequency
	<char>	<num>	<num>	<num>

CODE:

```
Airbnb_data <- read.csv(file="C:/Users/simon/Documents/3142_ass2/airbnb-Sydney.csv", header=TRUE, na.strings =
"N/A")
```

```
library(dplyr)
library(ggplot2)
library(glmnet)
library(pROC)
library(car)
library(stringr)
library(moments)
library(mgcv)
library(xgboost)
```

```
#Classification
#DATACLEANING
```

```
Cleansed_data <- Airbnb_data
```

```
#Handle NAs in categorial predictors
#host_is_superhost -- NA - not evaluated
Cleansed_data$host_is_superhost <- as.character(Cleansed_data$host_is_superhost)
Cleansed_data$host_is_superhost[Cleansed_data$host_is_superhost == ""] <- "not_evaluated"
Cleansed_data$host_is_superhost <- as.factor(Cleansed_data$host_is_superhost)
```

```
#host_response_time -- NA - never
Cleansed_data$host_response_time <- as.character(Cleansed_data$host_response_time)
Cleansed_data$host_response_time[is.na(Cleansed_data$host_response_time)] <- "never_responds"
Cleansed_data$host_response_time <- as.factor(Cleansed_data$host_response_time)
```

```
#host_response_rate and host_acceptance_rate -- remove ~900 nas introduced
table(Airbnb_data$host_acceptance_rate)
table(Airbnb_data$host_response_rate)
```

```
table(Airbnb_data$host_neighbourhood)
#DROP host_neighbourhood (72% of data missing)
```

```
#host_since --> create host_tenure_days
Cleansed_data$host_since <- as.Date(Cleansed_data$host_since)
```

```
#proxy when data was collected using most recent host creation date
snapshot_date <- max(Cleansed_data$host_since)
Cleansed_data$host_tenure_days <- as.numeric(snapshot_date - Cleansed_data$host_since)
```

```
#Host_verification count --> sum number of verification methods rather than string list
Cleansed_data$verification_count <- str_count(Cleansed_data$host_verifications, ",") + 1
```

```

#Amenity count --> sum number of unique amenities
Cleansed_data$amenity_count <- str_count(Cleansed_data$amenities, ",") + 1

#Property_type: Combine rare levels (observations < 10) into an "other" category
#may resolve issues with train/test data splits
property_type_counts <- table(Cleansed_data$property_type)
rare_property_types <- names(property_type_counts[property_type_counts < 10])

Cleansed_data$property_type <- as.character(Cleansed_data$property_type)
Cleansed_data$property_type[Cleansed_data$property_type %in% rare_property_types] <- "Other"
Cleansed_data$property_type <- as.factor(Cleansed_data$property_type)
table(Cleansed_data$property_type)

#ASSEMBLING TRAINING/TEST data -- remove certain predictors and separate data
review_score_vars <- c("review_scores_rating", "review_scores_accuracy",
                      "review_scores_cleanliness", "review_scores_checkin",
                      "review_scores_communication", "review_scores_location",
                      "review_scores_value")

drop_vars <- c("X", "id", review_score_vars, "bathrooms_text", "host_neighbourhood",
              "host_since", "host_verifications", "amenities",
              "first_review", "last_review", "host_acceptance_rate", "host_response_rate")

model_data <- Cleansed_data %>%
  select(-any_of(drop_vars)) %>%
  mutate(rating_q1 = as.factor(rating_q1))

model_data <- na.omit(model_data)
nrow(model_data); nrow(Airbnb_data)
#Preserved all obs

#Build entire model matrix first to avoid issues with test and training data not sharing identical columns

x_all <- model.matrix(rating_q1 ~ ., data = model_data)[, -1]
y_all <- model_data$rating_q1

#Split data into training and test
set.seed(3142)

train_index <- sample(seq_len(nrow(model_data)), size = 0.7 * nrow(model_data))
train_data <- model_data[train_index, ]
test_data <- model_data[-train_index, ]

#Matrix Model for glmnet
x_train <- x_all[train_index, ]
y_train <- y_all[train_index]
x_test <- x_all[-train_index, ]
y_test <- y_all[-train_index]

#LASSO regression model

```

```

set.seed(3142)
cv_lasso <- cv.glmnet(x_train, y_train, family = "binomial", alpha = 1, type.measure = "auc")
plot(cv_lasso)
cv_lasso$lambda.min
cv_lasso$lambda.1se

#RIDGE regression model
set.seed(3142)
cv_ridge <- cv.glmnet(x_train, y_train, family = "binomial", alpha = 0, type.measure = "auc")

#LASSO vs RIDGE regression
pred_lasso <- predict(cv_lasso, newx = x_test, s = "lambda.1se", type = "response")
pred_ridge <- predict(cv_ridge, newx = x_test, s = "lambda.min", type = "response")

roc_lasso <- roc(y_test, as.numeric(pred_lasso))
roc_ridge <- roc(y_test, as.numeric(pred_ridge))
table(Airbnb_data$property_type)
auc(roc_lasso)
auc(roc_ridge)

#TAKE THE COEFFS FROM THE LASSO REGRESSION
lasso_coefs <- coef(cv_lasso, s = "lambda.1se")
selected_vars <- rownames(lasso_coefs)[which(lasso_coefs != 0)]
selected_vars <- setdiff(selected_vars, "(Intercept)")
selected_vars

#Selected variables remove some property_types and neighbourhood_cleansed
#Majority of neighbourhoods are removed (37/38) -- remove neighbourhood_cleansed from model
#Many property_types retained (9/16) -- reintroduce all property_types

glm_formula <- rating_q1 ~ host_response_time +
  host_total_listings_count +
  property_type +
  maximum_nights +
  availability_90 +
  number_of_reviews_130d +
  instant_bookable +
  amenity_count +
  host_is_superhost +
  host_identity_verified +
  longitude +
  bedrooms +
  minimum_minimum_nights +
  number_of_reviews +
  estimated_occupancy_1365d +
  price_per_room

glm_with_neighbourhood <- update(glm_formula, .~. + neighbourhood_cleansed)

#glm with neighbourhood cleansed vs without

glm_v1 <- glm(glm_formula, data = train_data, family = "binomial")
glm_v2 <- glm(glm_with_neighbourhood, data = train_data, family = "binomial")

```

```
anova(glm_v1, glm_v2, test = "Chisq")
```

```
#Neighbourhood cleansed appears to be somewhat significant p - 0.02274 but increases DF by 37
```

```
glm_test_no_neigh <- predict(glm_v1, newdata = test_data, type = "response")
```

```
glm_test_with_neigh <- predict(glm_v2, newdata = test_data, type = "response")
```

```
auc(roc(test_data$rating_q1, glm_test_no_neigh))
```

```
auc(roc(test_data$rating_q1, glm_test_with_neigh))
```

```
#AUC decreases -- evidence of overfitting -- preserve glm_v1 without neighbourhood cleansed
```

```
summary(glm_v1)
```

```
#Take the exponent to convert log-odds into odds ratio
```

```
exp(coef(glm_v1))
```

```
#Highest OR is host_is_superhostt -- don't know what factors affect becoming a superhost
```

```
#Could be data leakage
```

```
#Intercept term is ^74 -- this is due to the longitude of sydney being across 150 - 151 degs
```

```
#Change: center both training and test data using training longitude mean
```

```
longitude_mean <- mean(train_data$longitude)
```

```
-----#COMMENTED TO NOT ACCIDENTALLY RUN AGAIN
```

```
#train_data$longitude <- train_data$longitude - longitude_mean
```

```
#test_data$longitude <- test_data$longitude - longitude_mean
```

```
-----#COMMENTED TO NOT ACCIDENTALLY RUN AGAIN
```

```
mean(train_data$longitude)
```

```
mean(test_data$longitude)
```

```
#Some property_types had high standard error and are insignificant
```

```
#-- Entire vacation home has only 13 obs --
```

```
#increase minimum obs for to be classified as "other" to 20
```

```
property_type_counts <- table(Cleansed_data$property_type)
```

```
rare_property_types <- names(property_type_counts[property_type_counts < 20])
```

```
Cleansed_data$property_type <- as.character(Cleansed_data$property_type)
```

```
Cleansed_data$property_type[Cleansed_data$property_type %in% rare_property_types] <- "Other"
```

```
Cleansed_data$property_type <- as.factor(Cleansed_data$property_type)
```

```
##(3/4) host_response_time were found to be insignificant -- like with neighbourhood_cleansed - remove
```

```
glm_no_response_time <- update(glm_v1, . ~ . - host_response_time)
```

```
anova(glm_no_response_time, glm_v1, test = "Chisq")
```

```
#0.2107 likelihood -- likely insignificant
```

```
pred_no_response_time <- predict(glm_no_response_time, newdata = test_data, type = "response")
```

```
auc(roc(test_data$rating_q1, pred_no_response_time))
```

```
auc(roc(test_data$rating_q1, glm_test_no_neigh))
```

```
#AUC decreases by 0.0018 -- remove host_response_times
```

```
vif(glm_v1)
```

```
#All vif numbers below 3.2 and gvif below 1.5 -- below 2 rule of thumb concern threshold
```

```
#multicollinearity is not a major issue in the model
```

```

#MODEL V3 - reperform lasso and ridge
model_data_v3 <- Cleansed_data %>%
  select(-any_of(drop_vars)) %>%
  mutate(rating_q1 = as.factor(rating_q1))

model_data_v3 <- na.omit(model_data_v3)
nrow(model_data_v3); nrow(Airbnb_data)

x_all_v3 <- model.matrix(rating_q1 ~ ., data = model_data_v3)[, -1]
y_all_v3 <- model_data_v3$rating_q1

#Split data into training and test
set.seed(3142)

train_index_v3 <- sample(seq_len(nrow(model_data_v3)), size = 0.7 * nrow(model_data_v3))
train_data_v3 <- model_data_v3[train_index_v3, ]
test_data_v3 <- model_data_v3[-train_index_v3,]

x_train_v3 <- x_all_v3[train_index_v3, ]
y_train_v3 <- y_all_v3[train_index_v3]
x_test_v3 <- x_all_v3[-train_index_v3, ]
y_test_v3 <- y_all_v3[-train_index_v3]

#train_data_v3$longitude <- train_data_v3$longitude - longitude_mean
#test_data_v3$longitude <- test_data_v3$longitude - longitude_mean
#commented to avoid rerunning

#LASSO regression model v3
set.seed(3142)
cv_lasso_v3 <- cv.glmnet(x_train_v3, y_train_v3, family = "binomial", alpha = 1, type.measure = "auc")
plot(cv_lasso_v3)
cv_lasso_v3$lambda.min
cv_lasso_v3$lambda.1se

#RIDGE regression model v3
set.seed(3142)
cv_ridge_v3 <- cv.glmnet(x_train_v3, y_train_v3, family = "binomial", alpha = 0, type.measure = "auc")

#LASSO vs RIDGE regression
pred_lasso_v3 <- predict(cv_lasso_v3, newx = x_test_v3, s = "lambda.1se", type = "response")
pred_ridge_v3 <- predict(cv_ridge_v3, newx = x_test_v3, s = "lambda.min", type = "response")

roc_lasso_v3 <- roc(y_test_v3, as.numeric(pred_lasso_v3))
roc_ridge_v3 <- roc(y_test_v3, as.numeric(pred_ridge_v3))

auc(roc_lasso_v3)
auc(roc_ridge_v3)
#results insignificantly different from before - likely from noise fluctuation
#Prior data manip. steps do not affect predictive accuracy and resolve instability

set.seed(3142)
lasso_coefs_v3 <- coef(cv_lasso_v3, s = "lambda.1se")
selected_vars_v3 <- rownames(lasso_coefs_v3)[which(lasso_coefs_v3 != 0)]

```

```
selected_vars_v3 <- setdiff(selected_vars_v3, "(Intercept)")
selected_vars_v3
```

```
#differences with prev lasso -- 2 Property types dropped (entireplace and private room in bed and breakfast)
#(small categories folded into other)
#Everything else is identical
```

```
glm_formula_v3 <- rating_q1 ~ host_total_listings_count +
  property_type +
  maximum_nights +
  availability_90 +
  number_of_reviews_130d +
  instant_bookable +
  amenity_count +
  host_is_superhost +
  host_identity_verified +
  longitude +
  bedrooms +
  minimum_minimum_nights +
  number_of_reviews +
  estimated_occupancy_1365d +
  price_per_room
```

```
glm_v3 <- glm(glm_formula_v3, data = train_data_v3, family = "binomial")
summary(glm_v3)
exp(coef(glm_v3))
vif(glm_v3)
```

```
#glm_v3 analysis
pred_glm_v3 <- predict(glm_v3, newdata = test_data_v3, type = "response")
```

```
roc_glm_v3 <- roc(test_data_v3$rating_q1, pred_glm_v3)
auc(roc_glm_v3)
plot(roc_glm_v3, main = "ROC curve - logistic regression (v3)")
```

```
#convert model to classification outputs using 0.5 threshold
pred_class <- ifelse(pred_glm_v3 > 0.5, 1, 0)
conf_matrix_model <- table(Predicted = pred_class, Actual = test_data_v3$rating_q1)
conf_matrix_model
```

```
accuracy_model <- sum(diag(conf_matrix_model)) / sum(conf_matrix_model)
precision_model <- conf_matrix_model["1","1"] / sum(conf_matrix_model["1",])
recall_model <- conf_matrix_model["1","1"] / sum(conf_matrix_model[, "1"])
accuracy_model
precision_model
recall_model
```

```
#Upon analysis of accuracy and precision data -- try to optimise code by looking at different thresholds
#0.25, 0.5, 0.75
```

```
#Function to compute at different thresholds
evaluate_threshold <- function(pred_probs, actual, threshold) {
  pred_class <- ifelse(pred_probs > threshold, 1, 0)
```

```

conf_matrix <- table(Predicted = factor(pred_class, levels = c(0,1)),
                     Actual = actual)

accuracy <- sum(diag(conf_matrix)) / sum(conf_matrix)
precision <- conf_matrix["1","1"] / sum(conf_matrix["1",])
recall <- conf_matrix["1","1"] / sum(conf_matrix[, "1"])

list(threshold = threshold, confusion_matrix = conf_matrix,
     accuracy = accuracy, precision = precision, recall = recall)
}

#Test thresholds
thresholds_to_test <- c(0.25, 0.5, 0.75)

threshold_results <- lapply(thresholds_to_test, function(t) {
  evaluate_threshold(pred_glm_v3, test_data_v3$rating_q1, t)
})
names(threshold_results) <- paste0("threshold_", thresholds_to_test)

#Build summary for thresholds
threshold_summary <- do.call(rbind, lapply(threshold_results, function(r) {
  data.frame(threshold = r$threshold, accuracy = r$accuracy,
             precision = r$precision, recall = r$recall)
}))
rownames(threshold_summary) <- NULL
threshold_summary

threshold_results$threshold_0.25$confusion_matrix
threshold_results$threshold_0.5$confusion_matrix
threshold_results$threshold_0.75$confusion_matrix

#Create naive model for comparison: always predicts majority class 0
pred_class_naive <- rep(0, nrow(test_data_v3))
conf_matrix_naive <- table(Predicted = factor(pred_class_naive, levels = c(0,1)),
                          Actual = test_data_v3$rating_q1)
conf_matrix_naive

accuracy_naive <- sum(diag(conf_matrix_naive)) / sum(conf_matrix_naive)
accuracy_naive

#Plot of drivers of rating drivers
#manually encode values

q2_drivers <- data.frame(
  predictor = c("Superhost status", "Instant bookable", "Number of reviews",
               "Host total listings count", "Bedrooms", "Amenity count",
               "Price per room", "Host identity verified"),
  odds_ratio = c(2.35, 0.73, 0.99, 0.997, 1.10, 1.008, 1.004, 0.58)
)

#Order by effect strength
q2_drivers$direction <- ifelse(q2_drivers$odds_ratio > 1, "Increases odds", "Decreases odds")
q2_drivers$predictor <- factor(q2_drivers$predictor,

```

```

levels = q2_drivers$predictor[order(abs(log(q2_drivers$odds_ratio)))]

ggplot(q2_drivers, aes(x = predictor, y = odds_ratio, fill = direction)) +
  geom_col(width = 0.65) +
  geom_hline(yintercept = 1, linetype = "dashed", color = "grey") +
  coord_flip() +
  scale_fill_manual(values = c("Increases odds" = "#2166AC", "Decreases odds" = "#B2182B")) +
  labs(
    title = "Key Predictors of Achieving a Top-Quartile Rating",
    x = NULL,
    y = "Odds Ratio (reference line at 1 = no effect)",
    fill = NULL
  ) +
  theme_minimal(base_size = 12) +
  theme(
    legend.position = "bottom",
    panel.grid.minor = element_blank(),
    plot.title = element_text(face = "bold", size = 13)
  )

```

#Regression -----

```

#Data Cleaning -- Q1 CODE NEEDS TO BE RUN FIRST --
#Q3 specific has _Q3
#Reuses Cleansed_data from Q1 section above (host_is_superhost,
#host_response_time, host_tenure_days, verification_count, amenity_count,
#and collapsed property_type are already built

#New excluded variables:
#Host_response_rate/Host_acceptance_rate -- Large no. of NAs (p1)
# Can introduce later

#host_neighbourhood -- Very large number of NAs

#bathrooms_text -- duplicate of number numeric bathrooms

#first_review, last_review

#host_since, host_verifications, ammenities -- replaced by numeric versions

drop_vars_Q3 <- c("X", "id", "bathrooms_text", "host_neighbourhood",
  "host_since", "host_verifications", "amenities",
  "host_response_rate", "host_acceptance_rate",
  "first_review", "last_review",
  "response_rate_missing_Q3", "acceptance_rate_missing_Q3")

model_data_Q3 <- Cleansed_data %>%
  select(-any_of(drop_vars_Q3))

nrow(model_data_Q3)
ncol(model_data_Q3)

```



```

#Split data into training and Test
#NOTE SEED 31423 - THIS IS INTENTIONAL
set.seed(31423)
train_index_Q3 <- sample(seq_len(nrow(model_data_Q3)), size = 0.7 * nrow(model_data_Q3))
train_data_Q3 <- model_data_Q3[train_index_Q3, ]
test_data_Q3 <- model_data_Q3[-train_index_Q3, ]

#START WITH GLM MODEL - LOG link (From assignment part 1)
#Also evidence from large skew in price per room (2.31)
#Skewness of log(price_per_room) becomes -0.02
skewness(Airbnb_data$price_per_room)
skewness(log(Airbnb_data$price_per_room))

#Create Baseline Lasso/ridge regression GLMs
#Entire Matrix Model

x_all_Q3 <- model.matrix(price_per_room ~ ., data = model_data_Q3)[, -1]
y_all_Q3 <- model_data_Q3$price_per_room

x_train_Q3 <- x_all_Q3[train_index_Q3, ]
y_train_Q3 <- y_all_Q3[train_index_Q3]
x_test_Q3 <- x_all_Q3[-train_index_Q3, ]
y_test_Q3 <- y_all_Q3[-train_index_Q3]

set.seed(31423)

cv_lasso_Q3 <- cv.glmnet(x_train_Q3, y_train_Q3, family = Gamma(link = "log"), alpha = 1, nfolds = 10)
plot(cv_lasso_Q3)
cv_lasso_Q3$lambda.min
cv_lasso_Q3$lambda.1se

set.seed(31423)

cv_ridge_Q3 <- cv.glmnet(x_train_Q3, y_train_Q3, family = Gamma(link = "log"), alpha = 0, nfolds = 10)

#Evaluate Ridge and Lasso using Test Data

pred_lasso_Q3 <- predict(cv_lasso_Q3, newx = x_test_Q3, s = "lambda.min", type = "response")
pred_ridge_Q3 <- predict(cv_ridge_Q3, newx = x_test_Q3, s = "lambda.min", type = "response")

#MSE FUNc
MSE <- function(data, prediction) {
  mean((data - prediction)^2)
}

(MSE_lasso_Q3 <- MSE(y_test_Q3, pred_lasso_Q3))
(MSE_ridge_Q3 <- MSE(y_test_Q3, pred_ridge_Q3))

#Naive Baseline -- predict mean(price per room) for everyone
naive_pred_Q3 <- mean(y_train_Q3)
(mse_naive_Q3 <- MSE(y_test_Q3, naive_pred_Q3))

```

```
#---- Introduction of host acceptance rate and host response rate: Clean the data to preserve 8000 obs
#convert string to numeric and remove percentage sign
```

```
Cleansed_data$host_response_rate <- str_remove(Cleansed_data$host_response_rate, "%")
Cleansed_data$host_response_rate <- as.numeric(Cleansed_data$host_response_rate)
```

```
Cleansed_data$host_acceptance_rate <- str_remove(Cleansed_data$host_acceptance_rate, "%")
Cleansed_data$host_acceptance_rate <- as.numeric(Cleansed_data$host_acceptance_rate)
```

```
#Create temp values for nas:
```

```
Cleansed_data$host_response_rate[is.na(Cleansed_data$host_response_rate)] <- -1
Cleansed_data$host_acceptance_rate[is.na(Cleansed_data$host_acceptance_rate)] <- -1
```

```
#Mark where missing data values are: 1 - missing, 0 - present -- #Creates new variable to test as well
Cleansed_data$response_rate_missing_Q3 <- ifelse(Cleansed_data$host_response_rate == -1, 1, 0)
Cleansed_data$acceptance_rate_missing_Q3 <- ifelse(Cleansed_data$host_acceptance_rate == -1, 1, 0)
```

```
drop_vars_Q3_v2 <- setdiff(drop_vars_Q3, c("host_response_rate", "host_acceptance_rate"))
```

```
model_data_Q3_v2 <- Cleansed_data %>%
  select(-any_of(drop_vars_Q3_v2))
model_data_Q3_v2 <- na.omit(model_data_Q3_v2)
nrow(model_data_Q3_v2)
#preserves all 8000 obs
```

```
#Create matrixes and train, test data split
```

```
x_all_Q3_v2 <- model.matrix(price_per_room ~ ., data = model_data_Q3_v2)[, -1]
y_all_Q3_v2 <- model_data_Q3_v2$price_per_room
```

```
set.seed(31423)
train_index_Q3_v2 <- sample(seq_len(nrow(model_data_Q3_v2)), size = 0.7 * nrow(model_data_Q3_v2))
train_data_Q3_v2 <- model_data_Q3_v2[train_index_Q3_v2, ]
test_data_Q3_v2 <- model_data_Q3_v2[-train_index_Q3_v2, ]
```

```
x_train_Q3_v2 <- x_all_Q3_v2[train_index_Q3_v2, ]
y_train_Q3_v2 <- y_all_Q3_v2[train_index_Q3_v2]
x_test_Q3_v2 <- x_all_Q3_v2[-train_index_Q3_v2, ]
y_test_Q3_v2 <- y_all_Q3_v2[-train_index_Q3_v2]
```

```
#Replace -1 placeholders with median values
```

```
response_rate_median <-
median(train_data_Q3_v2$host_response_rate[train_data_Q3_v2$response_rate_missing_Q3 == 0])
acceptance_rate_median <-
median(train_data_Q3_v2$host_acceptance_rate[train_data_Q3_v2$acceptance_rate_missing_Q3 == 0])
```

```
#apply to the model matrices (column names match variable names -- numeric)
```

```
x_train_Q3_v2[train_data_Q3_v2$response_rate_missing_Q3 == 1, "host_response_rate"] <- response_rate_median
x_test_Q3_v2[test_data_Q3_v2$response_rate_missing_Q3 == 1, "host_response_rate"] <- response_rate_median
x_train_Q3_v2[train_data_Q3_v2$acceptance_rate_missing_Q3 == 1, "host_acceptance_rate"] <-
acceptance_rate_median
x_test_Q3_v2[test_data_Q3_v2$acceptance_rate_missing_Q3 == 1, "host_acceptance_rate"] <-
acceptance_rate_median
```

```

#Refit LASSO/ridge Gamma GLM with the reintroduced variables
set.seed(31423)
cv_lasso_Q3_v2 <- cv.glmnet(x_train_Q3_v2, y_train_Q3_v2, family = Gamma(link = "log"), alpha = 1, nfolds = 10)
set.seed(31423)
cv_ridge_Q3_v2 <- cv.glmnet(x_train_Q3_v2, y_train_Q3_v2, family = Gamma(link = "log"), alpha = 0, nfolds = 10)

pred_lasso_Q3_v2 <- predict(cv_lasso_Q3_v2, newx = x_test_Q3_v2, s = "lambda.min", type = "response")
pred_ridge_Q3_v2 <- predict(cv_ridge_Q3_v2, newx = x_test_Q3_v2, s = "lambda.min", type = "response")

(MSE_lasso_Q3_v2 <- MSE(y_test_Q3_v2, pred_lasso_Q3_v2))
(MSE_ridge_Q3_v2 <- MSE(y_test_Q3_v2, pred_ridge_Q3_v2))

#compare directly against your existing baseline
MSE_lasso_Q3; MSE_lasso_Q3_v2
MSE_ridge_Q3; MSE_ridge_Q3_v2

#Proceed to fit GAM using train_data_Q3 and test_data_Q3

#Lasso slightly worse model than ridge but lasso is simpler
#Take variables from Q3 lasso coefficients -- reduces concavity and removes redundant predictors (6 different
min/max nights)
#has availability -- remove: 2 non-t options out of 8000 -- virtually
#Availability_days -- only keep Availability_90 - only one that mattered in q1 model

lasso_coefs_Q3 <- coef(cv_lasso_Q3, s = "lambda.min")
selected_vars_Q3 <- rownames(lasso_coefs_Q3)[which(lasso_coefs_Q3 != 0)]
selected_vars_Q3 <- setdiff(selected_vars_Q3, "(Intercept)")

predictor_names_Q3 <- setdiff(names(model_data_Q3), c("price_per_room", "has_availability", "rating_q1"))
matched_vars_Q3 <- sapply(selected_vars_Q3, function(v) {
  matches <- predictor_names_Q3[sapply(predictor_names_Q3, function(p) startsWith(v, p))]
  matches[which.max(nchar(matches))]
})
final_vars_Q3 <- unique(unlist(matched_vars_Q3))

redundant_nights <- c("minimum_minimum_nights", "maximum_minimum_nights",
  "minimum_maximum_nights", "maximum_maximum_nights",
  "minimum_nights_avg_ntm")
redundant_availability <- c("availability_30", "availability_60", "availability_365")

final_vars_Q3 <- setdiff(final_vars_Q3, c(redundant_nights, redundant_availability))
final_vars_Q3

#Build GAM - to smooth continuous predictors -- need > 10 unique values
#Linear/ parametric terms for factors

build_gam_formula <- function(vars, data, response = "price_per_room", k = 10) {
  terms <- sapply(vars, function(v) {

    if (is.numeric(data[[v]]) && length(unique(data[[v]])) > 10) {
      paste0("s(", v, ", k = ", k, ")")
    } else {

```

```

      v
    }
  })

  as.formula(paste(response, "~", paste(terms, collapse = " + ")))
}

gam_formula_Q3 <- build_gam_formula(final_vars_Q3, train_data_Q3)
gam_formula_Q3

gam_Q3 <- gam(gam_formula_Q3, data = train_data_Q3, family = Gamma(link = "log"))
summary(gam_Q3)
gam.check(gam_Q3)      #checks basis dimension (k) adequacy -- watch for low p-values suggesting k too small
concurvity(gam_Q3, full = TRUE) #GAM's analogue of VIF -- watch for values close to 1

#Evaluate on test set
pred_gam_Q3 <- predict(gam_Q3, newdata = test_data_Q3, type = "response")
(MSE_gam_Q3 <- MSE(y_test_Q3, pred_gam_Q3))

#compare against your existing baselines
MSE_lasso_Q3; MSE_ridge_Q3; MSE_gam_Q3

#MSE_gam_Q3 clearly the best model --
#Latitude and host_tenure days need fixing -- pvalues <<< 2e-16
#edfs close to 9 --> increase k to 20

gam_formula_Q3_v2 <- update(gam_formula_Q3, . ~ . - s(latitude, k=10) - s(host_tenure_days, k=10)
  + s(latitude, k=20) + s(host_tenure_days, k=20))
gam_Q3_v2 <- gam(gam_formula_Q3_v2, data = train_data_Q3, family = Gamma(link = "log"))
gam.check(gam_Q3_v2)

#increasing k seems to not have solved pvalues -- both latitude and host_tenure still <<< 2e-16
#Latitude and longitude model geographic location together -- model them bivariately
#Further increase k for host_tenure_days to 30

gam_formula_Q3_v3 <- update(gam_formula_Q3_v2, . ~ . - s(latitude, k=20) - s(longitude, k=10) -
  s(host_tenure_days, k=20)
  + s(latitude, longitude, k = 10) + s(host_tenure_days, k=30))

gam_Q3_v3 <- gam(gam_formula_Q3_v3, data = train_data_Q3, family = Gamma(link = "log"))
gam.check(gam_Q3_v3)
pred_gam_Q3_v3 <- predict(gam_Q3_v3, newdata = test_data_Q3, type = "response")
(MSE_gam_Q3_v3 <- MSE(y_test_Q3, pred_gam_Q3_v3))

#host_tenure_days resolved -- increase k for bivariate longitude latitude
gam_formula_Q3_v4 <- update(gam_formula_Q3_v3, . ~ . - s(latitude, longitude, k=10)
  + s(latitude, longitude, k = 30))

gam_Q3_v4 <- gam(gam_formula_Q3_v4, data = train_data_Q3, family = Gamma(link = "log"))
gam.check(gam_Q3_v4)
pred_gam_Q3_v4 <- predict(gam_Q3_v4, newdata = test_data_Q3, type = "response")
(MSE_gam_Q3_v4 <- MSE(y_test_Q3, pred_gam_Q3_v4))

```

#Proceed to XGboosting tree:

```
#Reuse training data from glms -- x_train_Q3, y_train_Q3, x_test_Q3, y_test_Q3
d_train_Q3 <- xgb.DMatrix(data = x_train_Q3, label = y_train_Q3)
d_test_Q3 <- xgb.DMatrix(data = x_test_Q3, label = y_test_Q3)
```

```
#Tune with cross-validation -- assume that price per room follows gamma distribution
set.seed(31423)
```

```
xgb_params <- list(
  objective = "reg:gamma",
  max_depth = 6,
  eta = 0.05,
  subsample = 0.8,
  colsample_bytree = 0.8
)
```

```
xgb_cv_Q3 <- xgb.cv(
  data = d_train_Q3,
  params = xgb_params,
  nrounds = 1000,
  nfold = 10,
  early_stopping_rounds = 20,
  verbose = 0
)
```

```
str(xgb_cv_Q3, max.level = 1)
names(xgb_cv_Q3)
```

```
head(xgb_cv_Q3$evaluation_log)
#Stops after 357 rounds - metric being tracked is test_gamma_deviance_mean
```

```
best_nrounds_Q3 <- which.min(xgb_cv_Q3$evaluation_log$test_gamma_deviance_mean)
best_nrounds_Q3 #337
```

```
set.seed(31423)
xgb_Q3 <- xgb.train(
  params = xgb_params,
  data = d_train_Q3,
  nrounds = best_nrounds_Q3
)
```

```
pred_xgb_Q3 <- predict(xgb_Q3, newdata = d_test_Q3)
(MSE_xgb_Q3 <- MSE(y_test_Q3, pred_xgb_Q3))
```

```
MSE_lasso_Q3; MSE_ridge_Q3; MSE_gam_Q3; MSE_xgb_Q3
#4244.57; 4221.918; 3469.924; 3264.435
```

```
#xgb improves MSE with untuned combination
#Search for better combinations
```

```
grid_Q3 <- expand.grid(
```

```

max_depth = c(3, 6, 9),
eta = c(0.01, 0.05, 0.1)
)

grid_results_Q3 <- data.frame()

for (i in 1:nrow(grid_Q3)) {
  cat("Running combination", i, "of", nrow(grid_Q3),
    "-- max_depth =", grid_Q3$max_depth[i],
    ", eta =", grid_Q3$eta[i], "\n")

  set.seed(31423)
  cv_fit <- xgb.cv(
    data = d_train_Q3,
    params = list(objective = "reg:gamma",
      max_depth = grid_Q3$max_depth[i],
      eta = grid_Q3$eta[i],
      subsample = 0.8, colsample_bytree = 0.8),
    nrounds = 1000, nfold = 10, early_stopping_rounds = 20, verbose = 0
  )

  best_iter <- which.min(cv_fit$evaluation_log$test_gamma_deviance_mean)
  best_dev <- min(cv_fit$evaluation_log$test_gamma_deviance_mean)

  grid_results_Q3 <- rbind(grid_results_Q3,
    data.frame(max_depth = grid_Q3$max_depth[i], eta = grid_Q3$eta[i],
      best_nrounds = best_iter, cv_deviance = best_dev))
}

grid_results_Q3 <- grid_results_Q3[order(grid_results_Q3$cv_deviance), ]
grid_results_Q3
#Best grid results -- max_depth 6, eta 0.05, nround 337 - original configuration

#Variable interpretation/ importance of best model
importance_Q3 <- xgb.importance(model = xgb_Q3)
head(importance_Q3, 15)

xgb.plot.importance(importance_Q3, top_n = 15)

#Large price outlier in train and test data could be causing rmse differences -- max 999 vs 787 from train test split
summary(y_train_Q3)
summary(y_test_Q3)
sd(y_train_Q3); sd(y_test_Q3)
max(y_train_Q3); max(y_test_Q3)

#Final models summary/ data representation
#Ridge

Q3_final_ridge <- cv_ridge_Q3
pred_ridge_train <- predict(Q3_final_ridge, newx = x_train_Q3, s = "lambda.min", type = "response")
pred_ridge_test <- predict(Q3_final_ridge, newx = x_test_Q3, s = "lambda.min", type = "response")

```

```

(train_rmse_ridge <- sqrt(MSE(y_train_Q3, pred_ridge_train)))
(test_rmse_ridge <- sqrt(MSE(y_test_Q3, pred_ridge_test)))

#Lasso
Q3_final_lasso <- cv_lasso_Q3
pred_lasso_train <- predict(Q3_final_lasso, newx = x_train_Q3, s = "lambda.min", type = "response")
pred_lasso_test <- predict(Q3_final_lasso, newx = x_test_Q3, s = "lambda.min", type = "response")

(train_rmse_lasso <- sqrt(MSE(y_train_Q3, pred_lasso_train)))
(test_rmse_lasso <- sqrt(MSE(y_test_Q3, pred_lasso_test)))

#GAM
Q3_final_gam <- gam_Q3
pred_gam_train <- predict(gam_Q3, newdata = train_data_Q3, type = "response")
pred_gam_test <- predict(gam_Q3, newdata = test_data_Q3, type = "response")

(train_rmse_gam <- sqrt(MSE(y_train_Q3, pred_gam_train)))
(test_rmse_gam <- sqrt(MSE(y_test_Q3, pred_gam_test)))

#XGBM
Q3_final_xgbm <- xgb_Q3

#Rerun bc quirk of xgb (external pointers)
d_train_Q3 <- xgb.DMatrix(data = x_train_Q3, label = y_train_Q3)
d_test_Q3 <- xgb.DMatrix(data = x_test_Q3, label = y_test_Q3)

pred_xgb_train <- predict(Q3_final_xgbm, newdata = d_train_Q3)
pred_xgb_test <- predict(Q3_final_xgbm, newdata = d_test_Q3)

(train_rmse_xgb <- sqrt(MSE(y_train_Q3, pred_xgb_train)))
(test_rmse_xgb <- sqrt(MSE(y_test_Q3, pred_xgb_test)))

model_comparison_Q3 <- data.frame(
  Model = c("LASSO (Gamma GLM)", "Ridge (Gamma GLM)", "GAM (Gamma, smooth terms)", "XGBoost"),
  Train_RMSE = c(train_rmse_lasso, train_rmse_ridge, train_rmse_gam, train_rmse_xgb),
  Test_RMSE = c(test_rmse_lasso, test_rmse_ridge, test_rmse_gam, test_rmse_xgb)
)
model_comparison_Q3$Overfit_Gap <- model_comparison_Q3$Test_RMSE - model_comparison_Q3$Train_RMSE
model_comparison_Q3$Test_MSE <- model_comparison_Q3$Test_RMSE^2

#naive baseline for % reduction column
naive_rmse_Q3 <- sqrt(mse_naive_Q3)
model_comparison_Q3$Pct_Reduction_vs_Naive <- round(
  (1 - model_comparison_Q3$Test_MSE / mse_naive_Q3) * 100, 1
)

model_comparison_Q3

#Importance plot:
#Data preparation
importance_plot_data <- head(importance_Q3, 15)
importance_plot_data$Feature <- factor(importance_plot_data$Feature,
  levels = rev(importance_plot_data$Feature))

```

```
#Plot
ggplot(importance_plot_data, aes(x = Feature, y = Gain)) +
  geom_col(fill = "#2166AC", width = 0.65) +
  coord_flip() +
  labs(
    title = "Top 15 Predictors by Importance (XGBoost)",
    x = NULL,
    y = "Gain"
  ) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  theme_minimal(base_size = 12) +
  theme(
    panel.grid.minor = element_blank(),
    plot.title = element_text(face = "bold", size = 13),
  )
```

```
#Model Summaries (Q3)
```

```
#RIDGE
```

```
ridge_coefs_final <- as.matrix(coef(Q3_final_ridge, s = "lambda.min"))
ridge_coefs_final
```

```
#LASSO
```

```
lasso_coefs_final <- as.matrix(coef(Q3_final_lasso, s = "lambda.min"))
lasso_coefs_final
```

```
#GAM
```

```
summary(Q3_final_gam)
```

```
#XGBM
```

```
importance_Q3_full <- xgb.importance(model = Q3_final_xgbm)
importance_Q3_full
```

```
cat("XGBoost final model specification:\n")
cat("Objective:", xgb_params$objective, "\n")
cat("max_depth:", xgb_params$max_depth, "\n")
cat("eta:", xgb_params$eta, "\n")
cat("subsample:", xgb_params$subsample, "\n")
cat("colsample_bytree:", xgb_params$colsample_bytree, "\n")
cat("nrounds:", best_nrounds_Q3, "\n")
```

```
#EXCEL CODE SUBMISSION FOR Q5 -----
```

```
library(openxlsx)
library(dplyr)
library(stringr)
library(xgboost)
```

```
test_raw <- read.xlsx("C:/Users/simon/Documents/3142_ass2/AirbnbTest.xlsx", sheet = 1, detectDates = TRUE)
wb <- loadWorkbook("C:/Users/simon/Documents/3142_ass2/AirbnbTest.xlsx")
nrow(test_raw) #confirm 3865 -- must not change
```



```

test_clean <- test_raw

#host_is_superhost
test_clean$host_is_superhost <- as.character(test_clean$host_is_superhost)
test_clean$host_is_superhost[
  is.na(test_clean$host_is_superhost) |
  test_clean$host_is_superhost %in% c("", "N/A", "NA")
] <- "not_evaluated"

#host_response_time
#NOTE: read.xlsx() has no na.strings argument (unlike the read.csv() call used for
#training), so literal "N/A" text in the spreadsheet comes in as the STRING "N/A",
#not a true NA. Catch that explicitly here, mirroring the training-side logic.
test_clean$host_response_time <- as.character(test_clean$host_response_time)
test_clean$host_response_time[
  is.na(test_clean$host_response_time) |
  test_clean$host_response_time %in% c("", "N/A", "NA")
] <- "never_responds"

#host_since -> host_tenure_days -- REUSE training's snapshot_date, do not recompute
test_clean$host_since <- as.Date(test_clean$host_since)
test_clean$host_tenure_days <- as.numeric(snapshot_date - test_clean$host_since)

#host_verifications -> verification_count
test_clean$verification_count <- str_count(test_clean$host_verifications, ",") + 1

#amenities -> amenity_count
test_clean$amenity_count <- str_count(test_clean$amenities, ",") + 1

#property_type -- apply the SAME rare-level list computed from TRAINING data
test_clean$property_type <- as.character(test_clean$property_type)
test_clean$property_type[
  is.na(test_clean$property_type) |
  test_clean$property_type %in% c("", "N/A", "NA")
] <- "Other"
test_clean$property_type[test_clean$property_type %in% rare_property_types] <- "Other"
#also catch any property_type value never seen during training at all
test_clean$property_type[!(test_clean$property_type %in% levels(Cleansed_data$property_type))] <- "Other"

#Sweep check: scan every other factor variable for literal N/A / NA / "" strings ----
#read.xlsx() will not have converted these to true NA the way read.csv(na.strings="N/A")
#did on the training side, so anything missed here would silently turn into NAs once
#we force factor levels to match training in step 3.
factor_vars <- names(Cleansed_data)[sapply(Cleansed_data, is.factor)]
for (v in intersect(factor_vars, names(test_clean))) {
  bad <- test_clean[[v]] %in% c("N/A", "NA", "") | is.na(test_clean[[v]])
  if (any(bad)) {
    cat(v, ":", sum(bad), "literal N/A/blank/NA values found -- confirm these are handled above\n")
  }
}

#Force EVERY factor variable to match training's exact levels to prevent model.matrix from creating
#mismatched dummy columns

```

```

for (v in intersect(factor_vars, names(test_clean))) {
  test_clean[[v]] <- factor(as.character(test_clean[[v]]), levels = levels(Cleansed_data[[v]]))
}

#check for NAs introduced by any unseen category (a value in test with no matching training level)
na_check <- sapply(test_clean[intersect(factor_vars, names(test_clean))], function(x) sum(is.na(x)))
na_check[na_check > 0] #INSPECT THIS -- any nonzero entries need a decision before proceeding
#> named integer(0)

#Select the same predictor columns as model_data_Q3 (minus price_per_room, which doesn't exist here) ----
test_predictors <- test_clean %>% select(-any_of(c(drop_vars_Q3, "price_per_room")))
test_predictors <- test_predictors %>% select(all_of(setdiff(names(model_data_Q3), "price_per_room")))

#check for any remaining NAs before building the matrix
sum(is.na(test_predictors))
#> 0

#FIX FOR has_availability
#Force character/logical predictor columns to carry TRAINING's levels ----
#model.matrix() infers factor levels from whatever values are actually present in the
#data passed to it. Step 3 only re-levelled columns that were already factors in
#Cleansed_data -- but some predictors (e.g. has_availability) are character/logical and
#near-constant in training (has_availability is "t" for 7998/8000 training rows, "f" for
#only 2). In a smaller test batch it's entirely possible every row is "t", leaving only
#1 level and causing model.matrix() to error ("contrasts can be applied only to factors
#with 2 or more levels"). Explicitly set levels from model_data_Q3 for every such column
#so model.matrix() always builds the same dummy-column structure as x_train_Q3.
char_log_vars <- names(model_data_Q3)[sapply(model_data_Q3, function(x) is.character(x) || is.logical(x))]
char_log_vars <- setdiff(char_log_vars, "price_per_room")

for (v in intersect(char_log_vars, names(test_predictors))) {
  train_levels <- sort(unique(as.character(model_data_Q3[[v]])))
  test_predictors[[v]] <- factor(as.character(test_predictors[[v]]), levels = train_levels)
}

sapply(test_predictors[intersect(char_log_vars, names(test_predictors))], function(x) sum(is.na(x)))

#Build model matrix and align columns EXACTLY to x_train_Q3 ----
x_test_official <- model.matrix(~ ., data = test_predictors)[, -1]

missing_cols <- setdiff(colnames(x_train_Q3), colnames(x_test_official))
extra_cols <- setdiff(colnames(x_test_official), colnames(x_train_Q3))
missing_cols #should ideally be empty or explainable
extra_cols #should be empty -- if not, factor levels didn't align correctly, go back to step 3

#pad any missing columns with 0 (a level present in training but absent from this test batch)
for (col in missing_cols) {
  x_test_official <- cbind(x_test_official, 0)
  colnames(x_test_official)[ncol(x_test_official)] <- col
}

#reorder to match training exactly
x_test_official <- x_test_official[, colnames(x_train_Q3)]

```

```
dim(x_test_official) #should be 3865 x 103 > 3865 103
```

```
#Predict using your FINAL, already-fitted model (no retraining) ----
```

```
d_test_official <- xgb.DMatrix(data = x_test_official)
```

```
final_predictions <- predict(Q3_final_xgbm, newdata = d_test_official)
```

```
length(final_predictions) #must equal 3865 - TRUE
```

```
sum(is.na(final_predictions)) #must be 0 - TRUE
```

```
summary(final_predictions) #sanity check: values should look like plausible dollar prices, not negative/absurd
```

```
#Write predictions into column AY of the ORIGINAL workbook, preserving everything else ----
```

```
writeData(wb, sheet = 1, x = "z5690607", startCol = "AY", startRow = 1)
```

```
writeData(wb, sheet = 1, x = final_predictions, startCol = "AY", startRow = 2)
```

```
saveWorkbook(wb, "AirbnbTest_z5690607.xlsx", overwrite = TRUE)
```